

SIMATS ENGINEERING
ASSESSMENT TOOL 2
ARTIFICIAL INTELLIGENCE (CSA1708)

Name: G. Ramani

Reg. No:192312279

1. Hospital Doctor–Shift Assignment Using Backtracking Search

Problem Understanding

Assign doctors D1, D2 and D3 to Morning (M), Afternoon (A) and Night (N), with exactly one doctor per shift and one shift per doctor.

Constraints

- $D1 \neq \text{Night}$.
- D2 must work before D3, using Morning < Afternoon < Night.
- Only one doctor can be assigned to each shift.
- $D3 \neq \text{Morning}$.
- Every doctor must receive exactly one shift.

CSP Formulation

Component	Definition
Variables	D1, D2, D3
Domains	{Morning, Afternoon, Night}
Constraints	$D1 \neq N$; $D3 \neq M$; $D2 < D3$; AllDifferent(D1,D2,D3)
Goal	A complete assignment satisfying every constraint.

Backtracking Search – Intermediate Steps and Constraint Checks

Step	Trial Assignment	Constraint Check	Decision
1	D1 = Morning	D1 is not Night; valid	Accept temporarily
2	D2 = Afternoon	D2 must be before D3, so D3 would need Night; no duplicate yet	Accept temporarily
3	D3 = Night	D3 is not Morning; $D2(\text{Afternoon}) < D3(\text{Night})$; all shifts unique	Accept – solution found

A valid branch is therefore: D1=Morning → D2=Afternoon → D3=Night.

Backtracking Logic

BACKTRACK(assignment):

if all doctors are assigned:

return assignment

select an unassigned doctor D

```

for each shift in {Morning, Afternoon, Night}:
    if assigning shift to D satisfies all current constraints:
        add (D = shift)
        result ← BACKTRACK(assignment)
        if result ≠ FAILURE:
            return result
        remove (D = shift) // backtrack
return FAILURE

```

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Verification and Justification

D1 is not on Night; D3 is not on Morning; D2 works before D3 because Afternoon < Night; all three shifts are different; and every doctor has exactly one shift. Hence all constraints are satisfied. Backtracking is suitable because it builds assignments incrementally and immediately rejects any partial assignment that violates a constraint.

2. 5×5 Robot Grid Search Using the Given Constraints

Problem Understanding

The robot must travel from Start S to Goal G in a 5×5 grid. It can move only Up, Down, Left or Right; every move costs 1; obstacle cells cannot be entered; and Manhattan distance is specified as the heuristic.

Important Observation About the Given Question

The question does not provide the actual coordinates of S, G or the obstacle cells. Therefore, an exact numerical node-expansion sequence, exact priority-queue contents, path and cost cannot be uniquely determined from the supplied data. The correct solution is to show the general search procedure and state how the result is obtained once the missing grid configuration is known.

Manhattan Distance

For a node $n=(x,y)$ and goal $G=(x_g,y_g)$:

$$h(n) = |x - x_g| + |y - y_g|.$$

This estimates the minimum number of horizontal and vertical moves to the goal when obstacles are ignored.

BFS Procedure

Because every legal move has equal cost 1, Breadth-First Search (BFS) finds an optimal path in terms of number of moves. BFS uses a FIFO queue rather than a priority queue.

```

BFS(S, G):
  Queue ← [S]
  Visited ← {S}
  Parent[S] ← NIL
  while Queue is not empty:
    n ← DEQUEUE(Queue)
    if n = G:
      return RECONSTRUCT_PATH(Parent, G)
    for each neighbor m of n in {Up, Down, Left, Right}:
      if m is inside 5×5 grid
        and m is not an obstacle
        and m is not Visited:
          Visited ← Visited ∪ {m}
          Parent[m] ← n
          ENQUEUE(Queue, m)
  return FAILURE

```

Step-by-Step Queue Update Pattern

Stage	Action	Queue / Frontier Update
1	Insert start node	[S]
2	Remove S and generate all legal unvisited neighbors	Append valid Up/Down/Left/Right neighbors
3	Expand nodes level by level	All nodes at depth d are processed before depth d+1
4	Ignore obstacles/out-of-grid/visited cells	They are not inserted
5	When G is first reached	Use Parent links to reconstruct S→...→G and stop

How Manhattan Heuristic Fits the Wording

The question also asks for Manhattan distance and 'priority queue updates'. Strictly, those features correspond to a heuristic best-first method such as A*, not ordinary BFS. If a priority queue is required, use A* with $f(n)=g(n)+h(n)$, where $g(n)$ is the number of moves from S and $h(n)$ is Manhattan distance. At each step, expand the node with the smallest f-value.

```

A*(S, G):
  PriorityQueue ← {(S, g=0, f=h(S))}
  while PriorityQueue is not empty:
    n ← remove node with minimum f
    if n = G: return path
    for each legal neighbor m:
      new_g ← g(n) + 1
      h(m) ← ManhattanDistance(m,G)
      f(m) ← new_g + h(m)
      insert/update m if this route is better

```

Optimal Path and Cost

For BFS, once the actual grid is supplied, the first shortest path to G is optimal because each move costs 1. If the reconstructed path contains k moves, Total Cost = $k \times 1 = k$. An exact value of k cannot be calculated without the positions of S, G and obstacles.

Justification

BFS is complete and optimal for an unweighted/equal-cost grid. Manhattan distance is admissible for four-direction movement and is appropriate if the intended algorithm is A*. This answer preserves the stated BFS requirement while correctly explaining the heuristic/priority-queue terminology in the question.

3. Autonomous Rescue Robot in a Partially Observable Disaster Environment

a) Constraint Analysis and Effect on Decision Making

Constraint	Effect on Robot Decision Making
Movement: Up, Down, Left, Right	Limits the successor states to adjacent traversable cells.
Blocked cells / obstacles	Robot must reject blocked successors and re-plan when newly sensed obstacles invalidate its route.
Adjacent-cell sensing only	The world is partially observable; the robot cannot safely assume unseen cells are free.
Normal movement cost = 1	Each safe movement contributes 1 to cumulative path cost.
Risky zone additional cost = 2	Entering a risky cell costs $1+2=3$, so a longer safe route may be cheaper/better than a short risky route.
Minimize total cost while navigating safely	Decisions must balance progress, cost and risk rather than simply minimizing geometric distance.
Dynamic knowledge / online search	The robot must sense, update its internal map, act, and re-plan repeatedly as new information becomes available.

Constraint Prioritization

Safety and traversability are hard constraints: the robot must not enter known blocked cells. Reaching the survivor is the goal. Among feasible routes, the robot should minimize cumulative movement/risk cost. Unknown cells require cautious sensing and dynamic re-planning.

b) Solution Approach: Online Search Agent with Uniform-Cost Re-planning

A suitable approach is an Online Search Agent that maintains an internal map and uses Uniform Cost Search (UCS) on the currently known map. UCS is appropriate because action costs are non-uniform: a normal move costs 1 while entry into a risky zone costs 3 in total.

Step	Robot Operation
1	Initialize current position S and an internal map with known/unknown cells.
2	Sense adjacent cells and classify observed cells as free, blocked or risky.
3	Update the internal knowledge/map with the new percepts.
4	Assign transition costs: normal free cell = 1; risky cell = 3; blocked cell = not traversable.
5	Run UCS on the currently known traversable map toward G or the best safe frontier leading toward G.
6	Select the lowest cumulative-cost safe next action.
7	Move one step, incur its cost, then sense again.
8	If a new obstacle/risk changes the planned route, update the map and re-run UCS.
9	Repeat sense → update → plan → act until G is reached.

Conceptual Pseudocode

ONLINE_RESCUE_AGENT(S, G):

```
current ← S
knowledge ← initial partial map
total_cost ← 0
while current ≠ G:
    percept ← SENSE_ADJACENT_CELLS()
    UPDATE(knowledge, percept)
    for each known traversable cell:
        cost ← 1
        if cell is risky:
            cost ← 3
    plan ← UCS(current, G, knowledge, costs)
    if no complete known path exists:
        choose safest/lowest-cost promising frontier action
    else:
        choose first action of plan
    EXECUTE(action)
    total_cost ← total_cost + action_cost
```

```

current ← resulting position
return SUCCESS, total_cost

```

c) Application of AI Concepts

AI Concept	Application
Intelligent Agent	The robot perceives through sensors, maintains knowledge, selects actions and acts through motors to achieve the rescue goal.
Rationality	A rational robot chooses the action expected to maximize successful safe rescue while minimizing cumulative cost/risk based on currently available information.
Environment Type	Partially observable because only adjacent cells are sensed; sequential because actions affect future states; dynamic/uncertain because knowledge and accessibility may change; mostly discrete because the grid/actions are discrete; cost-sensitive because actions have different costs.

PEAS View

PEAS Element	Rescue Robot
Performance	Reach/rescue survivor safely, minimize total cost/risk/time, avoid collisions/blocked cells.
Environment	Grid rooms, obstacles, risky zones, survivor location and changing/unknown accessibility.
Actuators	Movement system: Up, Down, Left, Right; rescue/communication mechanisms if available.
Sensors	Adjacent-cell obstacle/risk sensing, localization and other onboard sensing.

d) Justification of the Chosen Approach

A pure offline shortest-path method is insufficient because the robot initially has incomplete information and must discover obstacles while moving. An Online Search Agent is therefore essential for repeated sensing and knowledge updates. UCS is a strong local planning method because costs differ between normal and risky cells; it expands the lowest cumulative-cost option first and finds the least-cost route over the currently known graph when costs are non-negative. Combining online perception/re-planning with UCS therefore satisfies partial observability, dynamic knowledge, obstacle avoidance and cost minimization.

Final Conclusion

The rescue robot should operate as a rational model-based online agent: sense adjacent cells, update an internal map, use cost-aware search, execute one or a few safe actions, and re-plan whenever new information appears. This produces a feasible, adaptive and safety-conscious solution under the stated constraints.